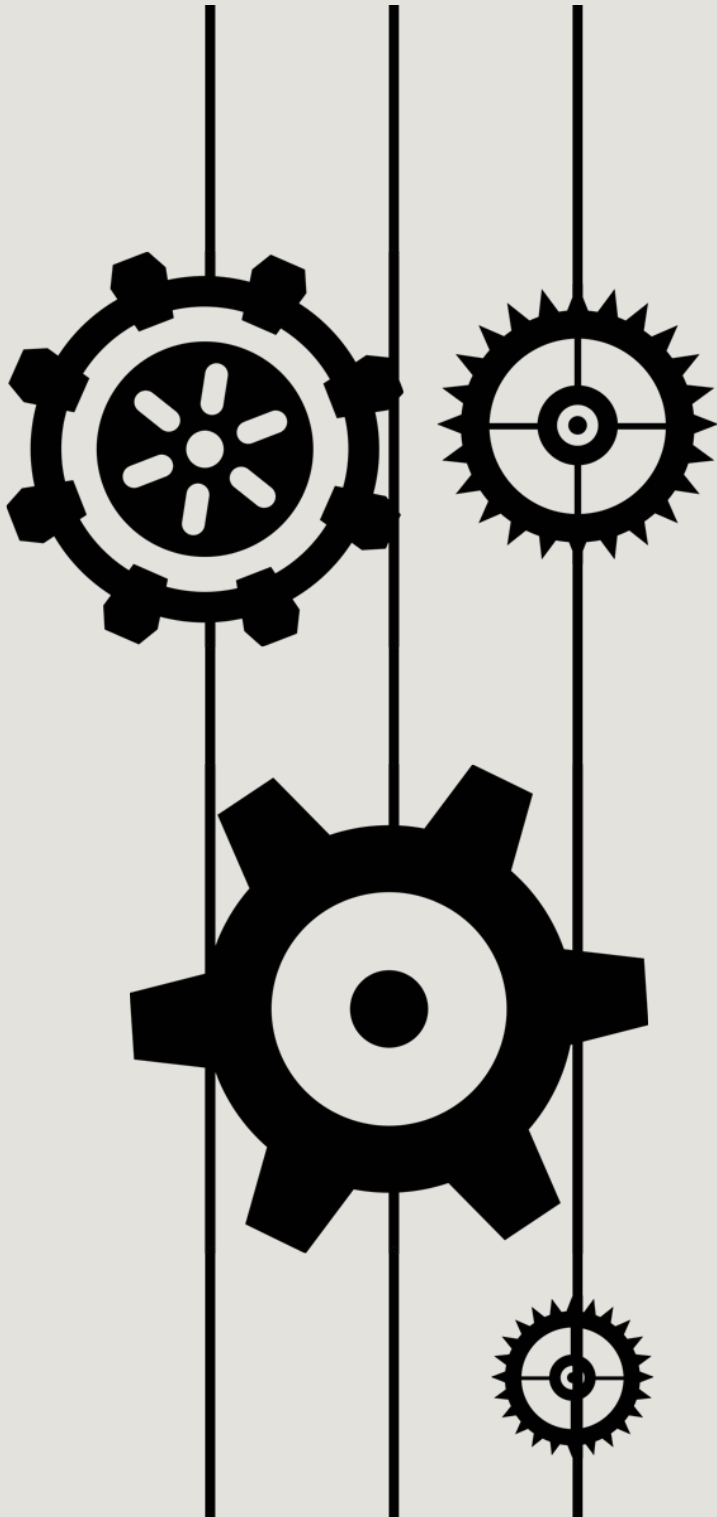


Predictive Analysis of Machine Failures



Nishant Kanwar
Roll no:- 3024030021
Msc. maths and computing
Submitted to:-
Dr. Jatin Bedi

CONTENT:-

1. Introduction	2
2. Dataset	3-6
3. Methodology	7-9
4. Results and discussion	10
5. Conclusion	10
6. Appendices	11-16

INTRODUCTION

1. Overview

In modern manufacturing, unexpected machine failures can lead to significant financial losses and production delays. Predictive Maintenance (PdM) uses sensor data and machine learning to foresee failures before they happen. Traditional classification methods predict just one type of failure, but in real situations, a machine can have more than one failure at once. Thus, this project focuses on a multi-label classification approach.

This work uses Random Forest and Gradient Boosting models wrapped in a MultiOutputClassifier to predict multiple failure types at the same time. The project compares how well the models perform and suggests the most effective one.

2. ProblemStatement

To develop a multi-label classification model that predicts specific machine failure types simultaneously using sensor data and machine parameters.

3. Goals

To preprocess and analyse sensor data from industrial machines.

To use Random Forest and Gradient Boosting to create multi-label classification models.

To assess and contrast the performance of the models.

To determine which machine failure prediction model performs the best.

4. Summary

This project aims to predict various types of machine failures by using sensor data and machine parameters. We employ a multi-label classification method to identify five different failure types: TWF, HDF, PWF, OSF, and RNF. We preprocess the dataset by removing unnecessary columns and one-hot encoding categorical data. Two machine learning models based on ensembles, Random Forest and Gradient Boosting, are used within a MultiOutputClassifier framework. We assess the models using precision, recall, and F1-score. The results indicate that Gradient Boosting performs better than Random Forest for most failure types. Thus, we recommend Gradient Boosting as the top model for predicting multiple machine failures. This approach can assist industries in using predictive maintenance and minimizing downtime.

DATASET

Description of the Dataset:

The dataset used in this project is an industrial machine failure dataset, which is commonly found on platforms like Kaggle and the UCI Machine Learning Repository. It includes sensor measurements and machine parameters that describe the operating conditions of manufacturing machines. The main goal of the dataset is to predict specific types of failure based on real-time sensor data.

Size and Structure of the Dataset:

Total rows (machine records): approximately 10,000

Ten columns total (features plus labels)

Type of data: numerical, tabular, and categorical

Each row shows a machine's current state.

Attributes in the dataset:

Feature	Description	Type
UDI	Unique identifier for each record (removed)	Integer
Product ID	Product type code (e.g., L, M, H series) (removed)	Categorical
Type	Machine type: L (Low), M (Medium), H (High)	Categorical
Air temperature [K]	Ambient air temperature in Kelvin	Numerical
Process temperature [K]	Temperature inside the process in Kelvin	Numerical
Rotational speed [rpm]	Machine spindle speed in revolutions/minute	Numerical
Torque [Nm]	Torque applied to the spindle in Newton-meters	Numerical
Tool wear [min]	Wear of cutting tool in minutes	Numerical

Target variables:

Label	Meaning
TWF	Tool Wear Failure
HDF	Heat Dissipation Failure
PWF	Power Failure
OSF	Overstrain Failure
RNF	Random Failure

Columns Removed and Their Reasons

UDI: Removed because it is only an IID and has no predictive value.

Product ID: Non-numeric, high cardinality, similar to type ID thus removed

Failure of the machine: overall failure flag (merely an OR of all failure types)
→ eliminated to prevent duplication

Types of Data

Numerical: Speed, torque, air temperature, process temperature, and tool wear

Categorical: Type (Type_L, Type_M, Type_H, converted to one-hot encoding)

TWF, HDF, PWF, OSF, and RNF are binary labels (0 or 1).

Preprocessing Performed

- Dropped unnecessary columns
- One-Hot Encoding for 'Type'
- Split into X (features) and Y (labels)
- Train-test split (80% / 20%)
- Checked feature correlation with heatmap
- No missing values in dataset (already clean)
- Minor class imbalance present

Features of the Dataset

Primarily numerical features → appropriate for models based on trees. Logical correlations are displayed by sensors (e.g., air temperature vs. process temperature). Certain labels are uncommon (for example, RNF occurs very infrequently). Prediction for rare classes is challenging due to the unbalanced failure distribution.

Challenges in Dataset

- Class imbalance
- Some failures overlap
- Hard to predict random failures

Dataset Summary

This dataset is perfect for creating a multi-label predictive maintenance model because it offers a rich combination of sensor-based numerical data and multiple failure labels. Because the dataset accurately depicts industrial conditions, machine learning models can discover connections between particular failure types and operating conditions. It is appropriate for both practical implementation and scholarly research due to its clear structure and relevance to real-world applications.

Dataset Snippet

UDI	Product ID	Type	Air temperature [K]	Process temperature [K]	Rotational speed [rpm]	Torque [Nm]	Tool wear [min]	Machine failure	TWF	HDF	PWF	OSF	RNF
1	M14860	M	298.1	308.6	1551	42.8	0	0	0	0	0	0	0
2	L47181	L	298.2	308.7	1408	46.3	3	0	0	0	0	0	0
3	L47182	L	298.1	308.5	1498	49.4	5	0	0	0	0	0	0
4	L47183	L	298.2	308.6	1433	39.5	7	0	0	0	0	0	0
5	L47184	L	298.2	308.7	1408	40	9	0	0	0	0	0	0
6	M14865	M	298.1	308.6	1425	41.9	11	0	0	0	0	0	0
7	L47186	L	298.1	308.6	1558	42.4	14	0	0	0	0	0	0
8	L47187	L	298.1	308.6	1527	40.2	16	0	0	0	0	0	0
9	M14868	M	298.3	308.7	1667	28.6	18	0	0	0	0	0	0
10	M14869	M	298.5	309	1741	28	21	0	0	0	0	0	0
11	H29424	H	298.4	308.9	1782	23.9	24	0	0	0	0	0	0
12	H29425	H	298.6	309.1	1423	44.3	29	0	0	0	0	0	0
13	M14872	M	298.6	309.1	1339	51.1	34	0	0	0	0	0	0
14	M14873	M	298.6	309.2	1742	30	37	0	0	0	0	0	0
15	L47194	L	298.6	309.2	2035	19.6	40	0	0	0	0	0	0
16	L47195	L	298.6	309.2	1542	48.4	42	0	0	0	0	0	0
17	M14876	M	298.6	309.2	1311	46.6	44	0	0	0	46	0	0
18	M14877	M	298.7	309.2	1410	45.6	47	0	0	0	0	0	0
19	H29432	H	298.8	309.2	1306	54.5	50	0	0	0	0	0	0
20	M14879	M	298.9	309.3	1632	32.5	55	0	0	0	0	0	0
21	H29434	H	298.9	309.3	1375	42.7	58	0	0	0	42	0	0
22	L47201	L	298.8	309.3	1450	44.8	63	0	0	0	0	0	0
23	M14882	M	298.9	309.3	1581	30.7	65	0	0	0	0	0	0
24	L47203	L	299	309.4	1758	25.7	68	0	0	0	0	0	0
25	M14884	M	299	309.4	1561	37.3	70	0	0	0	0	0	0
26	L47205	L	299	309.5	1861	23.3	73	0	0	0	0	0	0
27	L47206	L	299.1	309.5	1512	39	75	0	0	0	0	0	0
28	H29441	H	299.1	309.4	1811	24.6	77	0	0	0	0	0	0
29	L47208	L	299.1	309.4	1439	44.2	82	0	0	0	0	0	0
30	L47209	L	299	309.4	1693	30.1	84	0	0	0	0	0	0
31	M14890	M	299.1	309.5	1339	48.2	86	0	0	0	0	0	0
32	L47211	L	299	309.4	1798	25.5	89	0	0	0	0	0	0
33	L47212	L	299	309.4	1419	48.3	91	0	0	0	0	0	0
34	L47213	L	298.9	309.3	1665	32.5	93	0	0	0	0	0	0
35	M14894	M	298.8	309.1	1559	34.7	95	0	0	0	0	0	0
36	M14895	M	298.8	309.2	1452	48.6	98	0	0	0	48	0	0
37	M14896	M	298.9	309.2	1581	36.7	101	0	0	0	0	0	0
38	L47217	L	298.8	309.1	1439	39.2	104	0	0	0	0	0	0
39	H29452	H	298.9	309.2	1379	50.7	106	0	0	0	0	0	0
40	L47219	L	298.8	309.1	1350	52.5	111	0	0	0	0	0	0
41	L47220	L	298.8	309.1	1362	45.4	113	0	0	0	0	0	0
42	L47221	L	298.8	309.1	1368	50.8	115	0	0	0	0	0	0
43	M14902	M	298.8	309.1	1368	49.1	117	0	0	0	49	0	0
44	H29457	H	298.8	309.2	1372	48.5	120	0	0	0	0	0	0
45	M14904	M	298.8	309.1	1472	47.5	125	0	0	0	0	0	0
46	L47225	L	298.8	309.1	1489	49.1	128	0	0	0	0	0	0
47	M14906	M	298.7	309	1843	25.8	130	0	0	0	0	0	0
48	L47227	L	298.8	309.1	1418	46.3	133	0	0	0	0	0	0
49	H29462	H	298.8	309.2	1425	53.9	135	0	0	0	0	0	0
50	M14909	M	298.9	309.2	1412	44.1	140	0	0	0	0	0	0
51	L47230	L	298.9	309.1	2861	4.6	143	1	0	0	1	0	0
52	L47231	L	298.9	309.1	1383	54.9	145	0	0	0	0	0	0
53	H29466	H	298.8	309	1497	43.8	147	0	0	0	0	0	0
54	L47233	L	298.7	309	1565	35.1	152	0	0	0	0	0	0
55	L47234	L	298.7	309	1691	30.1	154	0	0	0	0	0	0
56	L47235	L	298.8	309.1	1512	40.7	156	0	0	0	0	0	0
57	L47236	L	298.8	309.1	1477	42.4	158	0	0	0	0	0	0
58	L47237	L	298.8	309.1	1513	40.3	160	0	0	0	0	0	0
59	M14918	M	298.8	309	1661	29.5	162	0	0	0	0	0	0
60	L47239	L	298.8	309.1	1378	54.4	165	0	0	0	0	0	0
61	L47240	L	298.8	309.1	1393	52.6	167	0	0	0	0	0	0
62	M14921	M	298.8	309	1592	38.2	169	0	0	0	0	0	0
63	L47242	L	298.8	309	1829	22.9	172	0	0	0	0	0	0
64	M14923	M	298.9	309	1812	25.9	174	0	0	0	0	0	0
65	L47244	L	299	309.1	1523	38.6	177	0	0	0	0	0	0
66	L47245	L	299	309.1	1394	45.1	179	0	0	0	0	0	0
67	M14926	M	298.9	308.9	1679	28.7	181	0	0	0	0	0	0
68	H29481	H	298.8	308.9	1466	44.1	184	0	0	0	0	0	0
69	L47248	L	298.8	308.9	1527	32.4	189	0	0	0	0	0	0
70	L47249	L	298.9	309	1410	65.7	191	1	0	0	1	1	0
71	M14930	M	298.9	309	1924	22.6	193	0	0	0	0	0	0
72	L47251	L	298.9	309.1	1452	45.5	196	0	0	0	0	0	0
73	L47252	L	298.9	309.1	1369	44.4	198	0	0	0	0	0	0
74	L47253	L	299	309.1	1592	35	200	0	0	0	0	0	0
75	L47254	L	298.9	309	1601	32.3	202	0	0	0	0	0	0
76	L47255	L	298.8	308.9	1379	46.7	204	0	0	0	0	0	0
77	L47256	L	298.8	308.9	1461	47.9	206	0	0	0	0	0	0
78	L47257	L	298.8	308.9	1455	41.3	208	1	1	0	0	0	0
79	L47258	L	298.8	308.9	1398	51.5	0	0	0	0	0	0	0
80	L47259	L	298.8	308.9	1402	37.9	2	0	0	0	0	0	0
81	H29494	H	298.8	308.8	1445	49.9	4	0	0	0	0	0	0

METHODOLOGY

Preparing Data

Pandas, a Python library, was used to import the dataset. Product ID and UDI (Unique Identifier), two non-informative columns, were eliminated. One-Hot Encoding was used to transform the categorical column Type (values L, M, and H) into numeric format.

Five distinct labels were used as outputs: TWF, HDF, PWF, OSF, and RNF. The target variable Machine failure was not included because it only represented the overall failure flag.

The resulting information was separated into:

X (features): every encoded and numeric variable Y

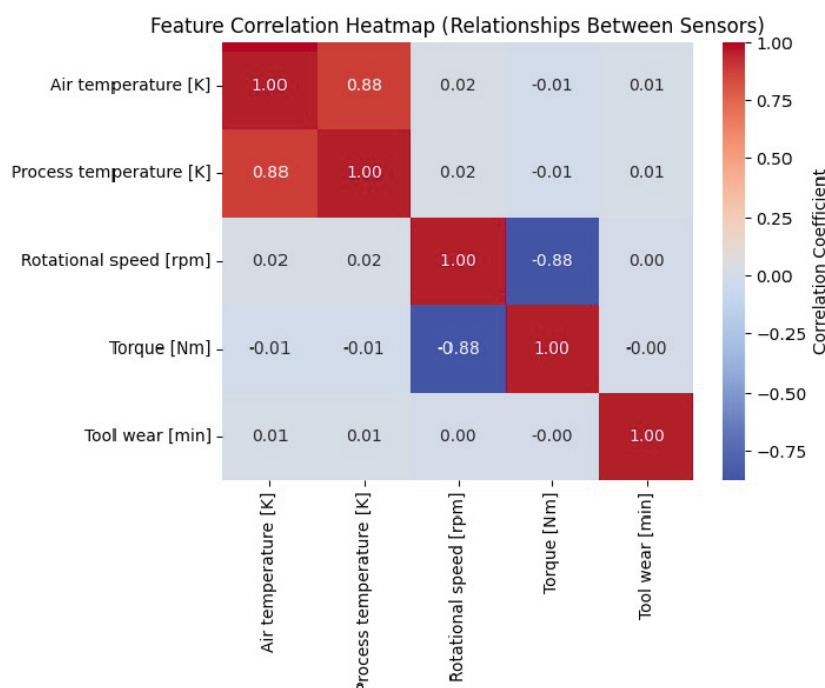
(targets): the five indicators of binary failure

For reproducibility, `train_test_split()` was used to divide the dataset into training (80%) and testing (20%) subsets using a fixed random seed (42).

Exploratory Data Analysis (EDA)

To comprehend the relationships between numerical variables like air temperature, process temperature, torque, rotational speed, and tool wear, exploratory analysis was done.

To find any linear dependencies, a correlation heatmap was created. No significant multicollinearity was discovered, and a moderate correlation between air and process temperature validated the sensor data's logical consistency.



Model Development

Because each record could have multiple simultaneous failures, the `MultiOutputClassifier` from Scikit-learn was employed. This wrapper allows a single estimator to predict multiple binary labels in parallel.

Two ensemble learning algorithms were trained and compared:

(a) Random Forest Classifier

```
n_estimators = 171  
max_depth = 0  
min_samples_leaf = 3.  
class_weight = "balanced"
```

Using bootstrapped samples, Random Forest creates an ensemble of decision trees and averages their results to minimise overfitting and variance.

(b) Classifier with Gradient Boosting

```
n_estimators = 100  
learning_rate = 0.1  
max_depth = 3.
```

Gradient Boosting builds trees one after the other, fixing any remaining mistakes from the previous trees. Despite being more computationally demanding, it typically achieves higher predictive accuracy.

The training set was used to train both models, and the test set, which was not visible, was used for evaluation.

Metrics for Evaluation

For every failure label, the model's performance was evaluated using common classification metrics:

Precision – proportion of correct positive predictions

Recall is the percentage of true positives that were correctly identified.

The F1-Score is the harmonic mean of recall and precision.

Support: the quantity of real samples in each class

To compare the two models, classification reports and summary tables were produced.

RESULTS AND DISCUSSION

Results

Failure Type	RF Recall	GB Recall	RF F1	GB F1	RF Precision	GB Precision
TWF	0.09	0.00	0.16	0.00	1.00	0.00
HDF	0.70	0.76	0.65	0.79	0.60	0.81
PWF	0.65	0.80	0.68	0.78	0.72	0.76
OSF	0.61	0.67	0.71	0.77	0.85	0.92
RNF	0.00	0.00	0.00	0.00	0.00	0.00

Comparison of Models

With limited recall on specific failure types, the Random Forest model produced results that were mediocre.

Stronger generalisation was demonstrated by the Gradient Boosting model, which consistently generated higher recall and F1-scores for HDF, PWF, and OSF labels.

Because RNF occurs so infrequently, both models had trouble detecting it.

Gradient Boosting was verified as the best model in the majority of metrics by a comparative heatmap of recall and F1-scores.

CONCLUSION

Choosing the Final Model

Gradient Boosting Classifier was chosen as the final model after a quantitative evaluation.

It successfully captured intricate feature interactions, showed a better balance between recall and precision, and offered consistent performance across a number of failure categories.

For upcoming predictive-maintenance applications involving multi-label failure detection, this model is thus advised.

Importing the libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.multioutput import MultiOutputClassifier
from sklearn.ensemble import RandomForestClassifier,
GradientBoostingClassifier
from sklearn.metrics import classification_report
```

Configuration (Model hyperparameters)

```
RANDOM_STATE = 42
TARGET_LABELS = ['TWF', 'HDF', 'PWF', 'OSF', 'RNF']
RF_BEST_PARAMS = {'n_estimators': 171, 'max_depth': None,
                  'min_samples_leaf': 3}
GB_PARAMS = {'n_estimators': 100, 'learning_rate': 0.1, 'max_depth':
3}
```

Data Preparation (Cleaning the data and defining features and targets)

```
# Load the dataset
df = pd.read_csv("machine_failure.csv")

# Drop unique IDs and the overall 'Machine failure' column
df_processed = df.drop(columns=['UDI', 'Product ID'])

# One-hot encode the 'Type' column (M, L, H)
df_processed = pd.get_dummies(df_processed, columns=['Type'],
drop_first=True)

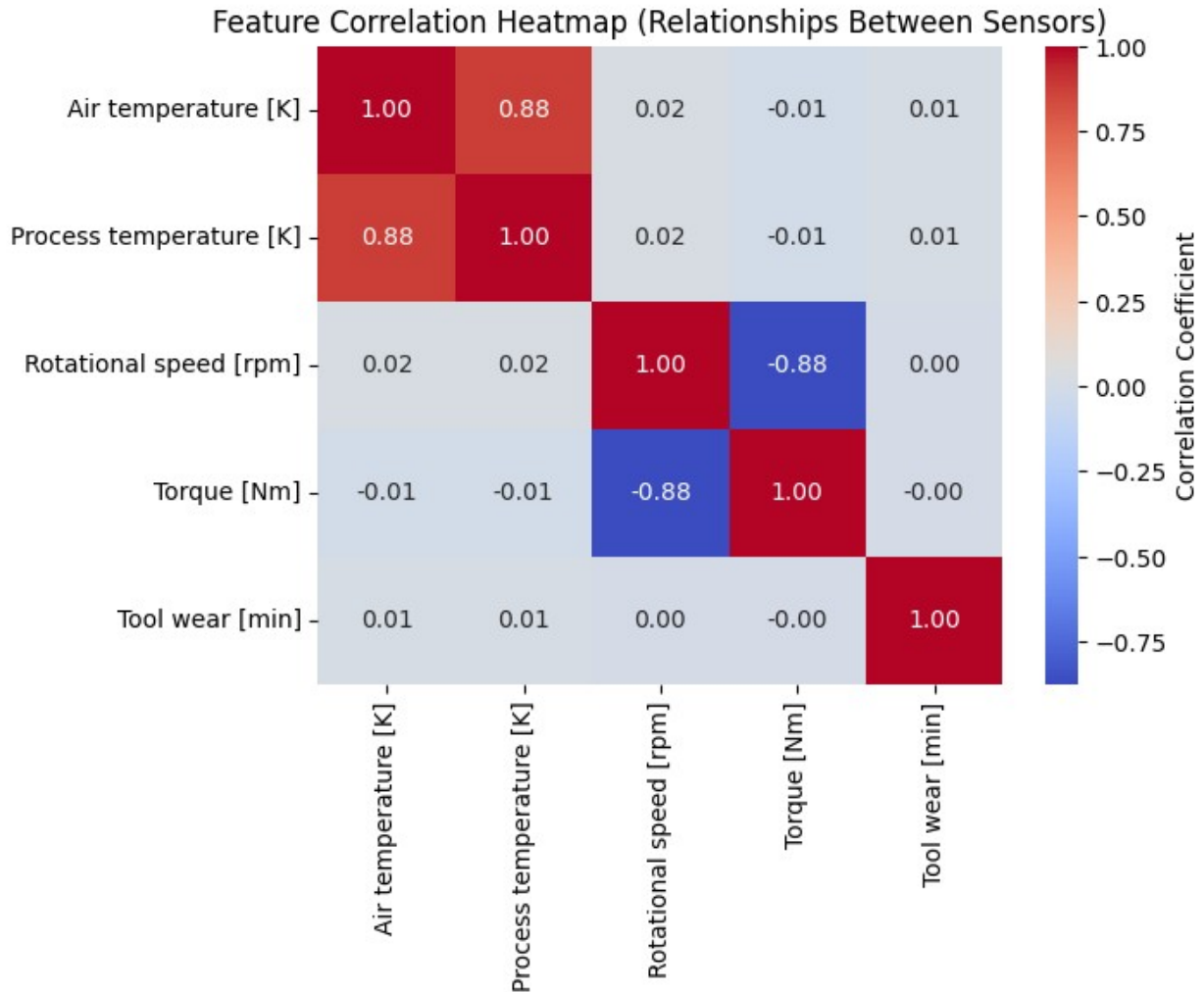
# Define features (X) and targets (Y)
Y = df_processed[TARGET_LABELS]
X = df_processed.drop(columns=TARGET_LABELS + ['Machine failure'])
```

Creating feature correlation heatmap

```
# Select numeric columns for correlation analysis
numeric_features = ['Air temperature [K]', 'Process temperature [K]',
                    'Rotational speed [rpm]', 'Torque [Nm]', 'Tool
wear [min]']
corr_matrix = df_processed[numeric_features].corr()

sns.heatmap(corr_matrix, annot=True, fmt=".2f", cmap='coolwarm',
```

```
cbar_kws={'label': 'Correlation Coefficient'})
plt.title('Feature Correlation Heatmap (Relationships Between Sensors)')
plt.show()
```



Splitting the data

```
X_train, X_test, Y_train, Y_test = train_test_split(X, Y,
test_size=0.2, random_state=RANDOM_STATE)
```

Model function and evaluation

```
#function created to train and evaluate different models (A. and B.)
and also return a failure classification report
def train_and_evaluate_model(base_classifier, model_name, X_train,
Y_train, X_test, Y_test):

    # Wrap the classifier for Multi-Label output
```

```

multi_target_model = MultiOutputClassifier(base_classifier)

# Train the model
multi_target_model.fit(X_train, Y_train)

# Predict and evaluate
Y_pred = multi_target_model.predict(X_test)

# making the report
report = classification_report(Y_test, Y_pred,
target_names=TARGET_LABELS,
                                output_dict=True, zero_division=0)

# Extract only the key failure metrics
report_df = pd.DataFrame(report).transpose() #making the transpose
failure_report = report_df.loc[TARGET_LABELS].copy() #using only
the required labels
return failure_report[['precision', 'recall', 'f1-score',
'support']]

# --- A. Tuned Random Forest Classifier (RF) ---
rf_classifier = RandomForestClassifier(
    n_estimators=RF_BEST_PARAMS['n_estimators'],
    max_depth=RF_BEST_PARAMS['max_depth'],
    min_samples_leaf=RF_BEST_PARAMS['min_samples_leaf'],
    random_state=RANDOM_STATE,
    class_weight='balanced'
)
rf_results = train_and_evaluate_model(rf_classifier, "Tuned Random
Forest", X_train, Y_train, X_test, Y_test)
print(rf_results)

# --- B. Gradient Boosting Classifier (GB) ---
gb_classifier = GradientBoostingClassifier(
    n_estimators=GB_PARAMS['n_estimators'],
    learning_rate=GB_PARAMS['learning_rate'],
    max_depth=GB_PARAMS['max_depth'], random_state=RANDOM_STATE
)
gb_results = train_and_evaluate_model(gb_classifier, "Gradient
Boosting", X_train, Y_train, X_test, Y_test)
print(gb_results)

```

	precision	recall	f1-score	support
TWF	1.000000	0.090909	0.166667	11.0
HDF	0.600000	0.705882	0.648649	17.0
PWF	0.722222	0.650000	0.684211	20.0
OSF	0.846154	0.611111	0.709677	18.0
RNF	0.000000	0.000000	0.000000	6.0
	precision	recall	f1-score	support
TWF	0.000000	0.000000	0.000000	11.0

HDF	0.812500	0.764706	0.787879	17.0
PWF	0.761905	0.800000	0.780488	20.0
OSF	0.923077	0.666667	0.774194	18.0
RNF	0.000000	0.000000	0.000000	6.0

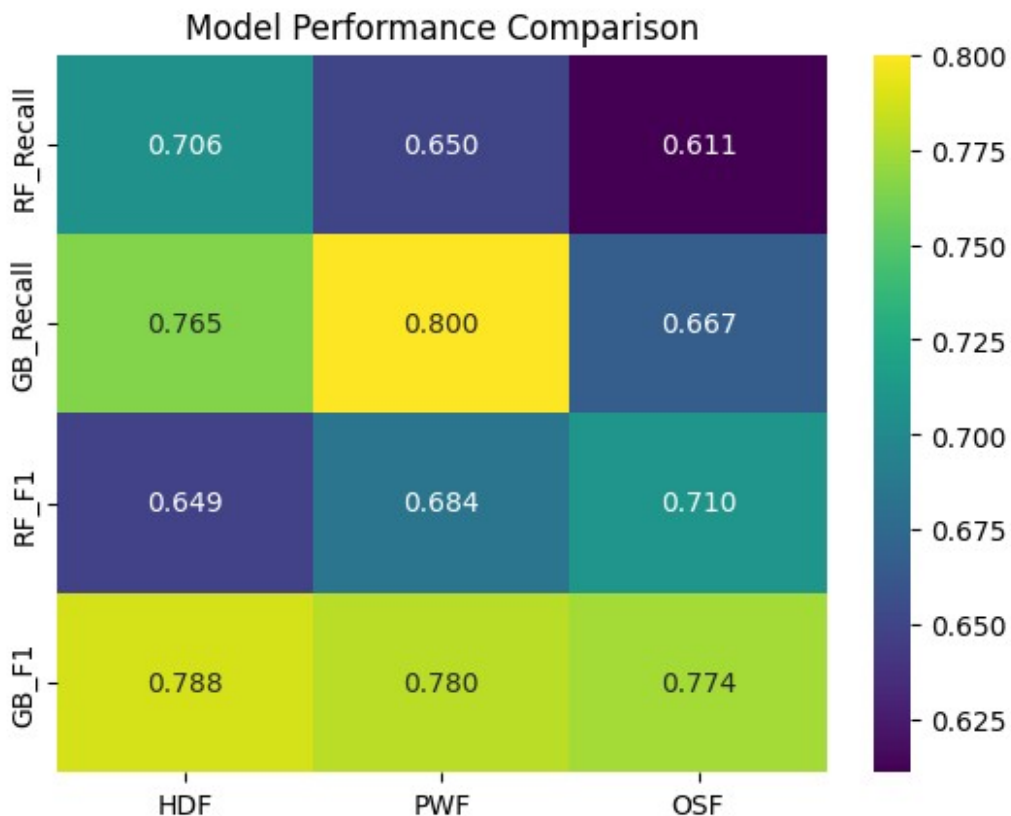
Final comparison and heatmap

```
# Combine results into one DataFrame
final_comparison_df = pd.concat(
    [rf_results.add_prefix('RF_'), gb_results.add_prefix('GB_')],
    axis=1
)

# Rearrange and simplify column names
final_comparison_df = final_comparison_df[['RF_recall', 'GB_recall',
'RF_f1-score', 'GB_f1-score', 'RF_precision', 'GB_precision',
'RF_support']]

# --- CREATE MODEL PERFORMANCE HEATMAP (Report Visual) ---
heatmap_data = pd.DataFrame({
    'RF_Recall': rf_results['recall'],
    'GB_Recall': gb_results['recall'],
    'RF_F1': rf_results['f1-score'],
    'GB_F1': gb_results['f1-score']
}).loc[['HDF', 'PWF', 'OSF']].T

# Plot heatmap
sns.heatmap(heatmap_data, annot=True, fmt=".3f", cmap='viridis')
plt.title('Model Performance Comparison')
plt.show()
```



Final Output

```
print("                          FINAL PROJECT OUTPUT AND COMPARISON")
print("\n--- PERFORMANCE COMPARISON: Random Forest and Gradient
Boosting ---")
print(final_comparison_df.round(4))
print("\nFinal Recommendation: Gradient Boosting is
the better model.")
```

FINAL PROJECT OUTPUT AND COMPARISON

```
--- PERFORMANCE COMPARISON: Random Forest and Gradient Boosting ---
RF_recall  GB_recall  RF_f1-score  GB_f1-score  RF_precision \
TWF        0.0909    0.0000      0.1667      0.0000      1.0000
HDF        0.7059    0.7647      0.6486      0.7879      0.6000
PWF        0.6500    0.8000      0.6842      0.7805      0.7222
OSF        0.6111    0.6667      0.7097      0.7742      0.8462
RNF        0.0000    0.0000      0.0000      0.0000      0.0000

GB_precision  RF_support
TWF           0.0000      11.0
HDF           0.8125      17.0
PWF           0.7619      20.0
OSF           0.9231      18.0
RNF           0.0000       6.0
```


Final Recommendation: Gradient Boosting is the better model.